

Learning Platform Project Briefing for Codex

1. Project Context

This project is a Laravel application with a React frontend template. The repository is public on GitHub and already initialized with some history.

The current codebase is still mostly a default Laravel + React setup. Several early concept files exist in folders named like `concept*`. These files contain broad ideas, sketches, and design directions for the platform. They should be read first to understand the intention behind the project.

The goal now is to stop staying purely theoretical and start building a first usable version. The implementation should still respect the concept, but it does not need to implement every idea immediately. The preferred approach is to create a first vertical slice that makes the core experience tangible.

The developer using this project is still learning PHP, Laravel, JavaScript, TypeScript, and React. Code should therefore be clean, readable, idiomatic, and include helpful English comments where they clarify intent, especially around framework-specific patterns.

2. Core Vision

This is a learning platform, but it should not feel like another Moodle, TryHackMe, Duolingo clone, or generic course platform.

The platform should invite users to explore knowledge through a configurable world, map, characters, activities, paths, and interactions. It should support curiosity, self-direction, competence, reflection, and meaningful progression.

The most important design goal is to create a learning environment that supports the psychological needs described by Self-Determination Theory:

- Autonomy: users should feel they have meaningful choices.
- Competence: users should feel they are improving and understanding more.
- Relatedness: users should feel connected to others in non-controlling, non-ranking ways.

The platform should avoid reward systems that make users learn mainly because they want points, badges, ranks, titles, loot, leaderboard positions, or other external incentives.

The ultimate goal is not to manipulate engagement. The goal is to create an environment where learning itself feels meaningful, playful, and satisfying.

3. Important Design Guardrails

The platform should avoid:

- Global points.
- Competitive leaderboards.
- Ranking users against each other.
- Titles or badges that become status symbols.
- Engagement-contingent rewards.
- Completion-contingent rewards that distract from learning.
- Rating other users.
- Social pressure mechanics.
- Dark patterns.
- Artificial streak pressure.

Allowed or encouraged instead:

- Personal progress reflection.
- Personal history.
- Personal bests where appropriate.
- Clear feedback that helps users understand what they learned.
- Optional exploration.
- Optional challenges.
- Cooperative or reflective features.
- User-owned notes.
- Message boards or shared notes that help future learners, without turning them into popularity contests.
- Unlocking new areas as a metaphor for growing understanding, not as a manipulative reward loop.

The platform may contain tests or questions, but they should be framed as tools for competence and reflection, not as judgment or ranking.

4. Domain-Agnostic and Configurable Storytelling

The platform should be as generic as possible regarding the learning domain.

A deployed instance should be configurable so it can fit different storytelling styles and subject areas.

Examples:

- A medieval learning platform could use castles, knights, forests, stone walls, taverns, scrolls, and old maps.
- An astronomy learning platform could use planets, stars, nebulae, dark space backgrounds, satellites, and space stations.
- A cybersecurity learning platform could use terminals, networks, servers, mythical systems, locked gateways, or digital landscapes.
- A biology learning platform could use cells, ecosystems, organisms, forests, or laboratories.

This means the platform should not hard-code one specific theme into the core logic.

The core system should separate:

- Learning structure.
- Map structure.
- Activity structure.
- Theme configuration.
- Visual assets.
- Text content.
- Conditions and progression logic.

Theme assets and UI visuals should be configurable where reasonable.

5. First Implementation Goal

Do not try to build the entire final platform at once.

The first useful version should be a vertical slice that demonstrates the core idea:

1. An admin or seeded configuration defines a small world map.
2. The user can see a hexagon tile map.
3. The user can hover over tiles and see a configurable edge highlight.
4. The user can select a tile/node.
5. The selected node opens a learning interaction.
6. The learning interaction can contain at least one NPC/dialogue-style activity.
7. The NPC activity supports multiple stages.
8. Each stage can have configurable text and optionally a different image.
9. A stage can lead to a question activity.
10. The user can answer a configurable multiple-choice question.
11. The result gives informational feedback, not controlling reward feedback.
12. Progress can be stored minimally.

This first version does not need full authentication complexity, full admin UI, complex permission systems, AI generation, multiplayer, guilds, deep nested maps, or every future mechanic.

A small but coherent slice is better than many unfinished abstractions.

6. Concept File Handling

Existing concept files should not simply be deleted.

Please create a new concept folder for the current consolidated concept, for example:

```
concept_current/
```

Inside it, create or maintain files such as:

- platform_vision.md
- sdt_design_principles.md
- mvp_scope.md
- domain_model.md
- activity_system.md
- map_system.md
- theme_system.md
- discarded_or_changed_ideas.md

Past ideas that changed should be documented with short notes explaining why they changed.

For example:

- “Linear activity sequence inside a node” changed to “node activity graph / multiple possible paths” because the newer design should allow more autonomy and richer learning encounters.
- “Global points” should be marked as rejected because they conflict with the SDT-oriented design goal.
- “Competitive leaderboards” should be marked as rejected because they create social comparison and controlling motivation.

The goal of concept files is not to endlessly theorize, but to keep the implementation aligned with the vision.

7. Hexagon World Map

The platform should include a world map made of hexagon tiles.

Each tile should be configurable. Possible tile properties:

- Position coordinates.
- Background image or visual style.
- Foreground image or object.
- Label.
- Node reference.
- Portal reference.
- Locked/unlocked state.
- Visibility condition.
- Hover highlight color.
- Hover highlight style.
- Optional tooltip.
- Optional theme-specific visual metadata.

Important interaction detail:

When the mouse hovers over a tile, the edges of the tile should be highlighted.

The highlight effect should be configurable, especially the color.

Rendering should be designed carefully. It may be better to separate tile layers conceptually and visually, for example:

- Base/background layer.
- Foreground/object layer.
- Overlay/highlight layer.
- Interaction layer.

The goal is to avoid unnecessary expensive re-rendering when only hover state changes.

Use React patterns that keep rendering reasonable. If a better approach exists, use it, but document the reasoning briefly.

8. Map Structure and Portals

The map system should eventually support more than one map.

A map can contain tiles.

Some tiles may represent portals to other maps.

This allows unlimited depth in theory:

- Main world map.
- Region map.
- City map.
- Building map.
- Room map.
- Inner symbolic map.
- Topic constellation map.

Not all of this needs to be implemented immediately, but the model should not make it impossible.

A portal tile should eventually be able to define:

- Target map.
- Visibility condition.
- Unlock condition.
- Display label.
- Visual asset.
- Optional transition text.

For the MVP, a simple map with normal learning tiles is enough, but the schema should be extensible.

9. Nodes and Activities

A map tile can point to a node.

A node is not just a single lesson page.

A node should represent a learning encounter that can contain one or more activities.

Previously, some concept files may describe nodes as containing multiple activities that are played sequentially. This should be revised.

The newer idea is that a node can contain multiple possible activity paths.

A node should eventually support an activity graph rather than only a linear list.

For example:

- Activity A leads to Activity B.
- Activity A can also lead to Activity C depending on user choice.
- Activity B leads to a question.
- The question can lead to different feedback stages.
- A later version could allow branching, optional side activities, or repeated practice.

For the first version, this can be implemented simply. It does not need a complex visual graph editor yet.

A practical MVP approach:

- Store activities belonging to a node.
- Store transitions between activities.
- Allow a default first activity.
- Allow simple next activity references.
- Keep the model flexible enough for branching later.

10. NPC / Dialogue Activity

One important activity type is an NPC interaction.

An NPC activity consists of multiple stages.

Each stage can define:

- NPC name.
- NPC image.
- Dialogue text.
- Optional background image.
- Optional mood/expression metadata.

- Optional forward button.
- Optional backward button.
- Optional next activity transition.
- Optional choices later.

The user should be able to move forward and backward through dialogue stages using arrows.

The NPC image should be configurable per stage. This allows a character to visually react, change expression, appear older/younger, become corrupted, become happier, etc.

A simple MVP NPC activity could work like this:

- Render NPC image.
- Render dialogue bubble.
- Render previous/next arrows.
- At the final stage, show a button to continue to the next activity.

11. Question Activity

Another important activity type is a question.

For the first version, support multiple-choice questions.

A question should define:

- Question text.
- Answer options.
- Which option or options are correct.
- Feedback for correct answer.
- Feedback for incorrect answer.
- Optional explanation.
- Optional next activity on correct answer.
- Optional next activity on incorrect answer.

The feedback should be informational.

Good feedback:

- "Correct. The important clue is that the failed login events come from many source IPs, which suggests credential stuffing rather than one user mistyping a password."
- "Not quite. Look again at the source IP distribution. A single IP would suggest a different investigation path."

Bad feedback:

- "You earned 50 points!"
- "You failed!"

- “You are better than 70% of users.”
- “Keep your streak alive!”

The point is to help the learner understand.

12. Progress Tracking

The first version should store minimal progress.

Useful progress data:

- Which nodes the user opened.
- Which activities the user reached.
- Which questions the user answered.
- The answer selected.
- Whether the answer was correct.
- Timestamp.
- Optional attempt count.

Avoid building complicated analytics too early.

Progress should mainly support:

- Returning to where the user left off.
- Showing what the user has explored.
- Supporting competence reflection later.

13. Admin and Content Creation

Eventually, admins should be able to create and configure:

- Maps.
- Tiles.
- Nodes.
- Activities.
- NPCs.
- Dialogue stages.
- Questions.
- Answer options.
- Themes.
- Assets.
- Conditions.
- Portals.

For the first version, a full admin UI is not required.

It is acceptable to use database seeders or simple config files to create the first demo world.

However, the code should be structured so an admin UI can be added later.

Avoid hard-coding demo content deep inside React components.

14. Theme and Asset System

The system should support configurable visuals.

Future theme configuration could include:

- Global background.
- Map background.
- Tile background image.
- Tile foreground image.
- Tile hover highlight color.
- Dialogue box style.
- Font choices.
- NPC images.
- Button style.
- Portal visuals.
- Locked tile visuals.
- Unlocked tile visuals.

For the MVP, theme configuration can be simple. It could be stored in a config file, JSON column, database table, or seeded data.

The important thing is to avoid designing the platform as if it only supports one fixed visual identity.

15. Technical Expectations

Use Laravel conventions where reasonable.

Use React and TypeScript cleanly.

Prefer readable code over clever code.

Add comments in English where they help a beginner understand what is happening.

Comments should explain why something exists, not repeat obvious code.

Good comment example:

```
// The hover state is kept outside the tile data so we do not mutate the  
configured map structure.
```

Bad comment example:

```
// Set x to 1
const x = 1;
```

Keep components reasonably small.

Avoid mixing too many responsibilities in one component.

Prefer clear names.

Prefer typed structures for map data, node data, activity data, and question data.

Use Laravel migrations and models for persistent domain objects.

Use seeders for demo content.

Use API resources or DTO-like structures if useful.

16. Database / Environment

A Docker container for the database exists.

Connection details are stored in `container.txt`.

Please inspect `container.txt` and check whether `.env` contains the correct database settings.

If `.env` is wrong, correct it.

Then run migrations.

If required, create migrations for the first domain model.

Potential early tables:

- `maps`
- `map_tiles`
- `learning_nodes`
- `activities`
- `activity_transitions`
- `npc_dialogue_stages`
- `questions`
- `question_options`
- `user_activity_progress`

- `themes` or `theme_settings`

This schema does not need to be final, but it should be clean enough to iterate on.

17. Suggested MVP Domain Model

A possible first model:

Map

Represents a world map or sub-map.

Fields could include:

- `id`
- `slug`
- `name`
- `description`
- `background_asset`
- `theme_config`

MapTile

Represents one hex tile on a map.

Fields could include:

- `id`
- `map_id`
- `q`
- `r`
- `label`
- `tile_type`
- `node_id`
- `target_map_id`
- `background_asset`
- `foreground_asset`
- `hover_color`
- `is_visible_by_default`
- `is_locked_by_default`
- `config_json`

Use axial coordinates `q` and `r` for hex grids unless there is a strong reason not to.

LearningNode

Represents a learning encounter opened from a tile.

Fields could include:

- id
- slug
- title
- description
- first_activity_id
- config_json

Activity

Represents one activity inside a node.

Fields could include:

- id
- node_id
- activity_type
- title
- sort_order
- config_json

Possible activity types:

- npc_dialogue
- multiple_choice_question
- text
- reflection
- external_task

Only `npc_dialogue` and `multiple_choice_question` are needed for the first version.

ActivityTransition

Represents movement between activities.

Fields could include:

- id
- from_activity_id
- to_activity_id
- condition_type
- condition_config

For MVP, simple unconditional transitions are enough.

NPCDialogueStage

Represents one stage in an NPC dialogue activity.

Fields could include:

- id
- activity_id
- stage_order
- npc_name
- npc_image
- dialogue_text
- background_asset
- config_json

Question

Represents a multiple-choice question activity.

Fields could include:

- id
- activity_id
- question_text
- explanation
- config_json

QuestionOption

Represents an answer option.

Fields could include:

- id
- question_id
- option_text
- is_correct
- feedback_text
- sort_order

UserActivityProgress

Represents minimal progress.

Fields could include:

- id

- user_id
- node_id
- activity_id
- question_id
- selected_option_id
- is_correct
- created_at

This is only a suggestion. Please adjust if Laravel conventions or implementation clarity suggest a better design.

18. Frontend UX Expectations

The first frontend should show the idea clearly.

Possible screens:

World Map Screen

- Shows hex tiles.
- Tiles are visually distinct.
- Hovering a tile highlights its edges.
- Clicking a tile opens the related node.
- Locked or inactive tiles can look different, even if lock logic is simple for now.

Node / Activity Screen

- Shows the current activity.
- For NPC dialogue:
 - Shows NPC image.
 - Shows dialogue bubble.
 - Shows previous/next arrows.
 - Allows moving through stages.
 - Continue button at the end.
- For question:
 - Shows question text.
 - Shows answer options.
 - Allows selecting an answer.
 - Shows feedback.
 - Allows continuing after feedback.

Optional Debug/Admin Screen

For early development, a simple debug/admin screen may be useful to inspect seeded maps, nodes, and activities.

Do not over-polish UI yet. A simple coherent version is better.

19. Tone and Feeling

The platform should feel:

- Calm.
- Curious.
- Exploratory.
- Meaningful.
- Playful without being manipulative.
- Like entering a world of knowledge.
- Like learning opens doors.
- Like mistakes are part of understanding.
- Like users are respected.

It should not feel:

- Like a casino.
- Like a productivity guilt machine.
- Like school grades.
- Like social ranking.
- Like a dopamine trap.
- Like a corporate LMS with a fantasy skin.

20. Future Ideas to Keep in Mind but Not Build Yet

Do not implement these unless they naturally fit the first slice.

Future ideas include:

- Nested maps through portals.
- Inventory.
- Tools that unlock obstacles.
- Key stones or symbolic unlocks.
- Player character panel.
- Customizable avatar/body.
- Notebook with Markdown export.
- Spaced repetition.
- AI-generated scenario questions.
- Admin AI provider configuration.
- Guilds or cooperative groups.
- World boss style cooperative challenges.
- Message boards for future learners.
- Topic constellations / night-sky knowledge map.
- Conditional tile visibility.
- Complex activity branching.
- VM/lab tasks.
- Reflection prompts.

- Content import/export.
- Theme marketplace or reusable theme packs.

These ideas should influence extensibility, but they should not block the MVP.

21. What to Do First

Please start by:

1. Reading the README.
2. Reading all concept folders/files.
3. Reading `container.txt`.
4. Checking `.env`.
5. Running the project locally if possible.
6. Creating or updating current concept summary files.
7. Proposing a concrete MVP implementation plan.
8. Implementing the first vertical slice:
 9. database schema,
 10. seed demo content,
 11. API endpoints,
 12. React map screen,
 13. React node/activity screen,
 14. NPC dialogue activity,
 15. multiple-choice question activity,
 16. minimal progress storage if feasible.

If the concept files conflict, prefer the newer direction described here:

- SDT-supportive design over reward-based gamification.
- Configurable generic platform over fixed theme.
- Activity paths/graphs over strictly linear node sequences.
- Informational feedback over points or grades.
- Small working vertical slice over broad unfinished architecture.

22. Development Style

Please work iteratively.

After inspecting the project, summarize what exists and what seems missing.

Then implement in small coherent steps.

Prefer a working foundation over perfect abstraction.

Avoid large rewrites unless necessary.

When adding files, keep naming clear.

When making assumptions, document them briefly.

When something is intentionally deferred, add a note to the current concept files or a TODO section.

The project should remain understandable for someone who is still learning the stack.